

## PRD: CAD Prediction Backend (FastAPI)

**Project:** Multimodal AI Hackathon 2026, Track A (Cardiovascular Risk Visualization & Prediction)

**Component:** FastAPI backend serving an existing trained XGBoost model

**Status:** Architecture and API contract. Implement only after reading the "Rules for Cursor" section.

---

### 0. Rules for Cursor (read first)

1. **Do NOT train, retrain, or modify any model.** Use the three provided artifacts as the source of truth.
  2. **Do NOT invent feature names, categories, ranges, or patient data.** Every feature fact must be read from the artifacts. If it can't be determined, mark it `NEEDS_CONFIRMATION` and ask.
  3. **Step 1 of implementation is inspection, not coding the API.** Write `scripts/inspect_artifacts.py`, run it, and print: metadata contents, encoder types and classes, model type and `classes_`, feature count and order, and library versions. Show the output to the user before continuing.
  4. The model must use **exactly 52 input features**, in the exact order stored in the metadata.
  5. `Cath`, `LAD`, `LCX` and `RCA` must **never** be accepted as prediction inputs.
  6. Probabilities must come directly from `predict_proba()`. Never fabricate or post-process them.
  7. **Dependency versions:** do not hardcode versions from memory. Verify the latest stable and security status of each package, and pin `xgboost`, `scikit-learn`, `pandas` and `numpy` to the **versions used in Colab** so the pickles load.
  8. Never commit secrets or `.env`. Commit `.env.example`.
- 

### 1. Goal

Expose the existing trained XGBoost CAD model through a small, reliable REST API that a React/Vite frontend can call to send 52 patient features and

receive a CAD / Normal prediction with probabilities.

**Users:** The React frontend (machine client). End users are clinicians and patients, through the frontend.

**Safety:** The product is decision support / educational only, not a diagnostic tool. The API response includes a disclaimer field so the frontend can always display it.

## 2. Scope

### In scope

- Load the 3 artifacts once at startup
- `GET /health`, `GET /model-info`, `POST /predict`
- Validation, categorical encoding, feature ordering, inference
- Standard error format, CORS, tests, deployment readiness

### Out of scope (do not build unless requested)

- Training or retraining
- User accounts, OAuth, JWT
- Database or patient-record storage
- Socket.io, email, cron
- Image analysis

## 3. Known gap: vessel-level predictions (needs confirmation)

The Track A PDF requires stenosis probabilities for **LAD, LCX and RCA** (for coloring the 3D arteries) and an interpretable breakdown (SHAP/LIME). The provided model predicts **overall CAD vs Normal only**.

- **Needs confirmation:** do vessel models exist?
- If not, they will be trained separately later on the same 52 features. The architecture below must allow adding more models without redesign (see section 12).
- This PRD implements the overall CAD model only. A `contributions` field is reserved in the response for later.

## 4. Architecture

```

React/Vite frontend
  ↓ JSON over HTTPS
FastAPI router (app/api)
  ↓
Schema validation (app/schemas)
  ↓
Preprocessing service (encoders, ordering)
  ↓
Model service (XGBoost predict_proba)
  ↓
JSON response → React frontend

```

- Artifacts load **once at app startup** into an in-memory registry (not per request).
- Routers handle HTTP only. Preprocessing and model logic live in `services/`.
- Feature definitions are **data-driven from the metadata**, never hardcoded in route code.

## 5. Artifacts

| File                                      | Purpose                                                 | Placement                              |
|-------------------------------------------|---------------------------------------------------------|----------------------------------------|
| <code>CAD_XGBoost_Model.pkl</code>        | Trained XGBoost model                                   | <code>backend/app/ml/artifacts/</code> |
| <code>CAD_XGBoost_Metadata.pkl</code>     | Feature names and order, target mapping, inference info | <code>backend/app/ml/artifacts/</code> |
| <code>CAD_Categorical_Encoders.pkl</code> | Categorical encoders from training                      | <code>backend/app/ml/artifacts/</code> |

Loading:

- Use `pickle` or `joblib`, whichever created each file. Detect, don't assume.
- Paths come from config ( `ARTIFACTS_DIR` ), not hardcoded strings.
- If any file is missing or fails to load, the app must fail startup loudly and `/health` must report `model_loaded: false` with HTTP 503.
- Pickle files execute code when loaded, so only load files created by this team.

#### Startup self-checks (fail the app if any fails):

1. Feature list from metadata has exactly **52** entries.
2. No excluded feature ( `Cath` , `LAD` , `LCX` , `RCA` ) is in the list.
3. Every categorical feature has an encoder.
4. Model's expected feature count equals 52.
5. Target mapping and `model.classes_` agree on which class is CAD and which is Normal.

## 6. Feature contract (52 features)

The exact list is **not hardcoded in this PRD**. It is read from

`CAD_XGBoost_Metadata.pkl`.

After inspection, Cursor must generate and fill this table (also saved as `docs/FEATURES.md`):

| #          | Feature       | Data type     | Categorical / Numeric | Allowed values / range                                                                                               | Required |
|------------|---------------|---------------|-----------------------|----------------------------------------------------------------------------------------------------------------------|----------|
| 1...<br>52 | from metadata | from metadata | from encoder presence | categorical:<br><code>encoder.classes_</code> ;<br>numeric:<br><code>NEEDS_CONFIRMATION</code><br>unless in metadata | Yes      |

Known from the training notebook: **18 categorical features** (per the encoders cell). Numeric ranges, missing-value handling and clinical bounds are `NEEDS_CONFIRMATION`.

**Excluded (never inputs):** `Cath` , `LAD` , `LCX` , `RCA` .

## 7. API design

All responses use this envelope.

Success:

```
{ "success": true, "data": { } }
```

Error:

```
{ "success": false, "error": { "code": "ERROR_CODE", "message":  
  "Human readable", "fields": { "feature": ["reason"] } } }
```

`fields` is present only on validation errors.

## 7.1 GET /health

Liveness and readiness. No auth.

```
{ "success": true, "data": { "status": "ok", "model_loaded": true }  
}
```

Returns 503 if artifacts are not loaded.

## 7.2 GET /model-info

Lets the frontend build its input form from the contract.

```
{ "success": true, "data": {  
  "model_name": "XGBoost CAD classifier",  
  "model_version": "<from metadata or file hash>",  
  "feature_count": 52,  
  "features": [  
    { "name": "<name>", "type": "categorical|numeric",  
      "allowed_values": ["..."], "required": true }  
  ],  
  "classes": ["Normal", "CAD"]  
}
```

Does not expose file paths, raw encoder internals or model internals.

## 7.3 POST /predict

Request:

```
{ "features": { "<feature_name>": "<value>" } }
```

- Categorical features are sent as **human-readable labels** (the values in `encoder.classes_`). The backend does the encoding.
- Numeric features are sent as numbers.
- Exactly the 52 model features. Unknown keys (including `Cath`, `LAD`, `LCX`, `RCA`) are rejected.

Response:

```
{ "success": true, "data": {
  "prediction": "CAD",
  "probabilities": { "CAD": 0.0, "Normal": 0.0 },
  "model_version": "...",
  "disclaimer": "For decision support and educational purposes
only. Not a substitute for formal diagnostic imaging."
} }
```

- `prediction` and class labels follow the metadata target mapping.
- Probabilities are the direct `predict_proba` output, mapped via `model.classes_` and the target mapping.
- Reserved for later (do not implement now): `vessels` (LAD/LCX/RCA probabilities), `contributions` (per-feature attribution).

## 8. Preprocessing flow

```
Frontend input
→ schema validation (exact keys, types)
→ categorical value check against allowed values
→ categorical encoding using saved encoders
→ numeric conversion (float)
→ reorder to exact metadata feature order
→ one-row pandas DataFrame (column names = feature names)
→ XGBoost predict_proba
→ map class probabilities to labels
```

Rules:

- Ordering is enforced with `df = df[feature_order]` after construction.
- Encoders are applied only to features the metadata/encoders mark as categorical.
- Never use `fit` or `fit_transform` at inference, only `transform`.
- Unseen categories must produce a 422, not a silent encoding.

## 9. Validation and error handling

| Case                                            | HTTP | Code              | Behavior                                                 |
|-------------------------------------------------|------|-------------------|----------------------------------------------------------|
| Missing features                                | 422  | VALIDATION_ERROR  | List all missing names at once                           |
| Unknown / excluded field                        | 422  | VALIDATION_ERROR  | Name the rejected field                                  |
| Invalid categorical value                       | 422  | VALIDATION_ERROR  | Include the allowed values                               |
| Non-numeric / NaN / infinity in a numeric field | 422  | VALIDATION_ERROR  | Name the field                                           |
| Wrong data type                                 | 422  | VALIDATION_ERROR  | Name the field                                           |
| Malformed JSON / bad body                       | 400  | INVALID_REQUEST   | Generic message                                          |
| Model or artifacts not loaded                   | 503  | MODEL_UNAVAILABLE | Generic message                                          |
| Unexpected error                                | 500  | INTERNAL_ERROR    | Generic message to client, real error logged server-side |

- Numeric range limits are enforced **only if** derived from metadata or training statistics. Otherwise `NEEDS_CONFIRMATION`.
- Missing-value handling (blank inputs) is `NEEDS_CONFIRMATION`. Default: all 52 required.

## 10. Security

**Expose to the frontend:** feature names, allowed categories, predictions and probabilities, model version, disclaimer.

**Never expose:** `.pkl` files, file paths, stack traces, raw encoder/model objects, environment variables.

Additional:

- Do not log patient feature values.

- Request body size limit.
- Rate limiting on `/predict` (optional for prototype).
- No authentication required (not specified in the requirements). Revisit if multi-user access is added.
- Run behind HTTPS in production.

## 11. CORS

- Development: allow `http://localhost:5173` (Vite default) via an env var.
- Production: allow only the deployed frontend origin via env var.
- No wildcard `*`. No credentials or cookies needed.
- Allowed methods: GET, POST, OPTIONS. Allowed header: Content-Type.
- Alternative for dev: Vite proxy to avoid CORS entirely.

## 12. Extensibility

- Models in an in-memory registry keyed by name ( `cad` , later `lad` , `lcx` , `rca` ).
- Feature definitions come from metadata, so adding features means updating artifacts, not route code.
- `/model-info` drives the frontend form.
- Adding vessel models later should add fields to the response, not break the existing contract.

## 13. Folder structure

```

backend/
├─ app/
│   └─ main.py                # app factory, startup, CORS,
error handlers
│   └─ api/
│       ├── health.py
│       ├── model_info.py
│       └─ predict.py
│   └─ core/
│       ├── config.py        # env-based settings
│       └─ errors.py         # error envelope + exception
handlers
│   └─ schemas/
│       ├── predict.py
│       └─ common.py

```

```

|   |   | services/
|   |   | | artifact_loader.py # loads + self-checks artifacts
|   |   | | preprocessing.py  # validate, encode, order
|   |   | | model_service.py   # predict_proba + label mapping
|   |   | |
|   |   | ml/
|   |   | | artifacts/
|   |   | | | CAD_XGBoost_Model.pkl
|   |   | | | CAD_XGBoost_Metadata.pkl
|   |   | | | CAD_Categorical_Encoders.pkl
|   |   | |
|   |   | scripts/
|   |   | | inspect_artifacts.py
|   |   | |
|   |   | tests/
|   |   | |
|   |   | docs/
|   |   | | PRD.md
|   |   | | FEATURES.md # generated from metadata
|   |   | |
|   |   | requirements.txt
|   |   | .env.example
|   |   | .gitignore
|   |   | README.md

```

## 14. Testing (before connecting the frontend)

1. Artifacts load; feature count is 52; no excluded feature present.
2. Every categorical encoder round-trips each of its classes.
3. **Golden test:** use one *real* row from the original dataset (provided by the team) and confirm the API `predict_proba` matches the Colab output for the same row.
4. Error cases return the correct code and envelope: missing field, extra field, `LAD` sent as input, bad category, bad number, wrong type, malformed JSON.
5. `/health` and `/model-info` respond as specified.
6. CORS preflight from `http://localhost:5173` succeeds.
7. Probabilities sum to ~1.

Do not create dummy patient data. Use real rows supplied by the team.

## 15. Deployment

- Python host with uvicorn (Render, Railway, Hugging Face Space or similar), preferably Docker.
- Pin `xgboost`, `scikit-learn`, `pandas`, `numpy` to Colab versions.
- Artifacts ship with the repo (Git LFS if large) or from private storage.

- Env vars: `ARTIFACTS_DIR`, `CORS_ORIGINS`, `ENV`, `LOG_LEVEL`.
- Health check path: `/health`.
- HTTPS enforced. Free tiers may sleep, so warm up before demos.
- Frontend uses `VITE_API_BASE_URL`.

## 16. Open items (must be resolved with the team)

1. Output of the artifact inspection (metadata, encoders, model type, library versions).
2. Do vessel models (LAD/LCX/RCA) exist?
3. Does the model accept missing values, or are all 52 features required?
4. Numeric valid ranges for each feature.
5. A real dataset row for the golden test.

## 17. Implementation order

1. Place artifacts, run `inspect_artifacts.py`, share the output.
2. Generate `docs/FEATURES.md` from the metadata, and get the team's confirmation.
3. Config, error envelope and app skeleton.
4. Artifact loader with startup self-checks.
5. Preprocessing service.
6. Model service.
7. `/health`, `/model-info`, `/predict`.
8. CORS.
9. Tests, including the golden test.
10. Dockerfile and deployment.